# Building a Blackjack Card Counting and Kelly Sizing Simulator: Engineering Notes

Elaine Wu

2026

## Overview

This is a writeup of the engineering process behind the Blackjack Card Counting & Kelly Sizing Simulator project on my portfolio site. The goal of the project was to simulate Hi-Lo card counting in blackjack and compare three betting strategies (flat betting, count-based spread betting, and Kelly-criterion sizing), using a simplified basic-strategy player and precomputed Monte Carlo simulation.

The interesting part of this project was not the initial build. It was a sequence of validation failures that surfaced once I started actually checking the output instead of trusting that the simulation logic was correct. Each failure pointed to a real, specific bug, and fixing one often exposed the next. I'm documenting that sequence here because I think the debugging process is more representative of real quantitative work than a clean writeup of a simulation that worked on the first try.

## Architecture

The simulation runs as a standalone TypeScript script, separate from the Next.js app that displays the results. The pipeline has three stages:

1. **Engine** ( `engine.ts` , `shoe.ts` , `strategy.ts` , `betting.ts` , `dealer.ts` ): simulates individual hands of blackjack against a shoe of cards, tracking the running count and true count via the Hi-Lo system, applying a simplified basic-strategy heuristic for player decisions, and sizing bets according to one of three strategies.

2. **Generation** ( `generate.ts` ): runs the engine across a fixed grid of configurations and hand-count checkpoints (1,000, 10,000, 100,000, and 1,000,000 hands), using a seeded random number generator so results are reproducible, and writes the output to static JSON files.

3. **Presentation** (the Next.js page and chart components): imports the precomputed JSON and renders it as charts and summary statistics. The simulation never runs live in the browser; it runs once, offline, and the site displays the result.

This separation mattered later. When the data turned out to be wrong, I could fix it at the source and regenerate, without touching anything in the presentation layer.

### Bug 1: Ruin Was Not Being Detected

The first time I ran the full simulation, every betting strategy ended with a final bankroll at or near zero after a million hands, with very large max-drawdown figures. My first instinct was that the bankroll was simply too small for the bet sizes involved.

To check, I added temporary logging that printed bankroll, true count, and bet size every 100,000 hands. The output told a different story. Bankroll hit zero by roughly hand 100,000, and then stayed at exactly zero (or, for the Kelly strategy, a small negative number) for the remaining 900,000 hands, with bet size frozen at zero the entire time.

The bug was not that the bankroll was too small. It was that the simulation had no concept of a stopping condition. Once a player goes broke, they cannot place further bets, so the loop correctly stopped betting, but it kept iterating anyway, logging the same dead state nine hundred thousand more times. This meant two things were wrong: the vast majority of "simulated" hands in a long run were never actually played, and the summary statistics (max drawdown, edge realized) were being computed over the full nominal hand count rather than the hands that had actually occurred.

The fix had three parts: break out of the hand loop the moment bankroll reaches zero or below, clamp losses so bankroll cannot go negative in the first place (a real player cannot lose more money than they have), and add explicit `handsActuallyPlayed` and `wentBust` fields to the output so the discrepancy between hands requested and hands played would be visible going forward rather than silently absorbed into the statistics.

### Bug 2: The Simplified Strategy Was Giving Up Too Much Edge

After fixing the ruin-detection bug, the underlying problem became visible rather than hidden: every strategy, including flat betting, was busting through its bankroll within tens of thousands of hands. Real basic-strategy blackjack carries a house edge of roughly 0.5% in a six-deck game. A bankroll of a thousand base bets should comfortably survive far more than 50,000 to 100,000 hands at that edge. Something was off by an order of magnitude.

To isolate the cause, I wrote a temporary diagnostic script that ran a flat-betting simulation with an artificially enormous bankroll, large enough that it could not realistically go bust, which let me measure the true house edge directly without ruin cutting the sample short. That measurement came back at $-1.78\%$, more than three times the expected figure.

The diagnostic broke results down by decision category (hard totals in the 12–16 range, doubled-down hands, split hands, and soft totals), and the worst offender was immediately obvious: hands where the player stood on a hard 12 through 16 were losing at a rate of $-20\%$. My simplified strategy heuristic had been standing on those totals only against a dealer upcard of 4, 5, or 6, and hitting against everything else, including dealer upcards of 2 and 3, where real basic strategy says to stand on 13 through 16. That single narrow rule was bleeding most of the excess edge.

I also reviewed the engine code directly rather than assuming the bug was purely a strategy-table gap. This confirmed that blackjack payouts, double-down logic, and split-hand settlement were all implemented correctly; the edge gap was a strategy quality issue, not an engine bug.

The fix expanded three specific decision rules: standing on 12 against a 4 through 6 and on 13 through 16 against a 2 through 6 (rather than 4 through 6 only), splitting a wider set of pairs against weak dealer upcards, and standing on soft 18 rather than only soft 19 and above. Re-running the same isolation diagnostic afterward showed the edge had moved to $-0.81\%$, within a realistic range for a deliberately simplified, non-exhaustive strategy table.

### Bug 3: A Measurement Bug Was Inflating the Apparent Edge

During the same diagnostic pass, a second, unrelated bug surfaced. The code computed realized edge as net profit divided by the sum of each hand's *initial* bet. For a hand that was doubled down or split, the actual amount wagered is two or more times the initial bet, so this denominator was systematically too small, which made every edge figure look more negative than it actually was.

The fix added an explicit `totalWagered` field to each hand's result, computed as the true total amount risked on that round including doubles and splits, and updated the edge calculation to use that figure instead of the initial bet. This was a pure bookkeeping fix with no effect on the actual gameplay logic, but it mattered: without it, every downstream statistic, including the per-strategy summary stats and the edge-vs-true-count chart, would have understated the player's real edge.

### Bug 4: Kelly Sizing Grew Without Bound

With the strategy and measurement fixes in place, I reran the full production data generation. The flat and spread strategies looked sensible. The Kelly strategy did not: over a million hands, its bankroll grew from a starting \$10,000 to roughly \$330 billion.

This is not a logic error in the conventional sense. Kelly betting sizes each wager as a fraction of current bankroll, so on a genuinely positive-edge game, bankroll grows multiplicatively, and compounding that over a million hands at speeds typical of advantage play produces an enormous number. The actual bug was an omission: the simulation had a cap limiting Kelly bets to a percentage of bankroll, but no fixed-dollar table maximum, which every real casino enforces regardless of how large a player's bankroll is. Without that ceiling, nothing in the simulation prevented bet sizes, and therefore bankroll growth, from running away once the bankroll itself became large.

The fix added a fixed table-maximum bet (set at fifty times the base bet, a realistic spread limit) that applies independently of the bankroll-percentage cap, with the more restrictive of the two governing the final bet size. After this fix, Kelly's bankroll at the million-hand checkpoint settled to roughly \$1.38 million, still the best-performing strategy by realized edge, but bounded by a constraint that reflects how the game is actually played in a real casino rather than an idealized, unconstrained one.

### Bug 5: The Edge Curve Had a Composition Problem

The last validation step was checking the edge-versus-true-count data that feeds the project's main chart. Plotting average outcome against true count revealed a clear anomaly: true count

+2, with over 200,000 hands of data, showed a worse edge than both true count +1 and true count 0, despite having one of the largest sample sizes in the entire dataset. A sample that large should not behave this noisily if the underlying relationship between true count and edge is real.

Investigating this surfaced two compounding causes, neither of which was a problem with the true-count calculation itself (which used ordinary floating-point division with no rounding artifacts).

First, the bucket boundaries were created by rounding true count to the nearest integer, so the +2 bucket absorbed every hand with a raw true count between 1.5 and 2.49, including a weak sub-range just above 1.5 that performed noticeably worse than the rest of the bucket and dragged its average down.

Second, and more significant: the edge curve had been computed as a simple round-by-round win-or-lose average, pooled across all three betting strategies. Spread and Kelly bet far more heavily at high true counts than flat betting does, so pooling all three strategies together biased each bucket toward whichever strategy happened to wager the most at that particular count, rather than reflecting the underlying game's edge. Recomputing the same +2 bucket using dollar-weighted outcomes (net result divided by total wagered) instead of a round-by-round average changed its value from $-1.50\%$ to $+0.11\%$, which is the actual signal hiding underneath a flawed aggregation method.

The fix rebuilt the edge curve to use only flat-strategy hands (the cleanest reference, since flat bets the same amount regardless of count) and to weight by dollars wagered rather than by hand count. After this change, the anomaly disappeared, and the curve formed a smooth, monotonic-ish relationship between true count and edge in both directions.

### A Smaller Refinement: Confidence Intervals

Even after the aggregation fix, the curve had visible noise at extreme true counts—for example, a dip at true count +5, where the sample size drops to a few thousand hands. Rather than treating this as a flaw to hide, I added a 95% confidence interval to each bucket, computed from the standard error of the per-hand dollar-weighted outcome within that bucket. The chart now displays these as a shaded band around the central estimate.

The bands widen sharply at extreme true counts, where sample sizes are smallest, and narrow to a tight range near true count zero, where the sample is largest. This turns a feature that could look like a bug (the +5 dip) into a feature that is honest about its own uncertainty: the chart now visually communicates which parts of the curve are well-supported and which are not, rather than presenting every point estimate with equal, false confidence.

### What I'd Take Away From This

The strategy I used throughout this project was to treat any number that looked wrong as a signal to investigate rather than a nuisance to patch around. Several of these bugs would have been easy to paper over: I could have raised the starting bankroll to stop the busting without ever finding the strategy-edge bug underneath it, or capped the edge curve's $y$-axis to hide the +2 anomaly without ever finding the aggregation bug that caused it. Each time, isolating the

actual mechanism, often with a small temporary diagnostic script built specifically to measure one thing in isolation, was what let me distinguish a real bug from an artifact of how I was looking at the data.

The simulation is still a simplification. The basic-strategy heuristic omits surrender, re-splitting, and some upcard-specific nuance; the Kelly edge estimate is a linear approximation rather than a precise advantage-play formula; and a million simulated hands represents years of continuous play, useful for showing long-run convergence, not a realistic individual session. I think being explicit about those simplifications, both here and in the methodology section of the project itself, is part of doing this kind of work honestly.